ASSIGNMENT 2 – CO2

ARTIFICIAL INTELLIGENCE – CSA17

CO2 Coverage: Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimisation problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

Problem Context – Autonomous Campus Shuttle

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

- Predict how the three algorithms will behave under different map conditions
- Explain the algorithmic reason for your prediction
- Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Nodes expanded and nodes generated
- Maximum frontier size
- Path cost and whether the path is optimal
- Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

Design Decision

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

Evidence Expected: Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

Problem Context – Exploration Rover in an Unmapped Facility

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

Your Challenge

Design experiments to investigate the online agent under:

- Targets at different distances from the start
- Unreachable targets
- Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

Changed Condition

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

Evidence Expected: Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

Problem Context – Game Engine for a Board Game

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

Your Challenge

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs)
2. One game tree deliberately engineered so that almost no cutoffs occur
3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

- Predict the expected number of nodes visited
- Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs.

Inquiry

Compare prediction with experimental result. Explain:

- Why one tree requires substantially more work than another
- Which characteristics of move ordering create or prevent cutoffs
- What this reveals about evaluating an algorithm using only the branching factor and depth

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

Problem Context – Turn-Based Strategy Game

A game AI must choose its next move by looking ahead in a game with an average of b legal moves per turn.

Your Challenge

Design a depth-limited minimax solution with an evaluation function of your own.

Before implementation, determine:

- How many positions are examined for lookahead depth d
- How the evaluation function is used when the search is cut off
- How the number of terminal evaluations changes with d

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

- Number of nodes expanded and evaluation-function calls
- Execution time, where useful
- Quality of the move chosen (for example, win rate over 20 games against a random player)

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

Evidence Expected: Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

Problem Context – University Exam Timetabling

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

Design Starting Point

You are given only the following principle:

A value may be removed from a variable's domain if some neighbouring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

Your Challenge

Convert this principle into a computational procedure. Determine:

- How the constraints are represented
- How a value without support is detected
- Which other pairs must be re-examined after a value is removed
- What condition indicates that the problem has no solution
- How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A timetable instance in which propagation solves or detects failure without any search
- A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

Correctness Inquiry

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

Problem Context – Delivery Tour Planning

A courier must visit n customers exactly once and return to the depot, minimising the total travel distance.

Your Challenge

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

Correctness Check: Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

Growth Investigation

Before experimentation, derive/predict how the number of candidate tours grows with n. Use at least three problem sizes to validate your prediction experimentally.

Local Search Alternative

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

Inquiry

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

Deployment Decision

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest n for which you would deploy exhaustive search, and the point at which you would switch to local search.

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

B. Define the Design Objective – 2 Marks

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

D. Predict Before Implementing – 1 Mark

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

E. Implement and Validate the Redesign – 2 Marks

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

F. Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

MARK DISTRIBUTION – ASSIGNMENT 2

Task	**Assessed Area**	**Marks**
Task 1	Uniform-cost, Greedy and A* search	6
Task 2	Online search agent	4
Task 3	Alpha-beta pruning	6
Task 4	Minimax and evaluation functions	6
Task 5	Constraint propagation and CSP	7
Task 6	Exhaustive and local search, optimisation	8
Task 7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Tasks 1–7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

Assessment Verification

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.